Machine Learning A (2026)
Home Assignment 1

Hannibal von Scholten (gmj118)

Contents

1 Make Your Own (10 points) **2**

2 Digits Classification with K Nearest Neighbors (40 points) **3**
2.1 Task #1 3

3 Regression (50 points) **6**
3.1 Task 1 (5 points) 6
3.2 Task 2 (10 points) 7
3.3 Task 3 (10 points) 7
3.4 Task 4 (5 points) 8
3.5 Task 5 (10 points) 9
3.6 Task 6 (10 points) 9

1 Make Your Own (10 points)

1. I would look at the students grades from completed courses and score from 1-10 of the difficulty of math in the course. The input sample space contains the different levels of math difficulty:

$$\mathcal{X} \in \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}.$$

2. The output/label space $\mathcal{Y}$ would be the grade in that course:

$$\mathcal{Y} \in \{-3, 00, 02, 4, 7, 10, 12\}.$$

3. The loss function could be defined by the defined as the squared error loss:

$$\ell(y, \hat{y}) = (y - \hat{y})^2.$$

   This means that larger errors are more harshly penalized compared to smaller errors.

4. To apply the $K-NN$ algorithm. I would define the distance function as the euclidean norm of x and x':

$$d(x, x') = ||x - x'||.$$

   In this case it's just the numerical difference between x and x':

$$d(x, x') = |x - x'|.$$

5. To test the performance of the $K - NN$ algorithm (hypothesis) h, i would divide the available data $S = \{(y_1, x_2), (y_2, x_2), \ldots, (x_N, y_N)\}$ into to sets, a training set consisting of the first n elements in S and a test set consisting of the last $N - n$ elements in S. I would the calculate the empirical risk of h on S:

$$\begin{aligned}\mathcal{R}_S(h) &= \frac{1}{N} \sum_{i=N-n}^{N} \ell(y_i, h(x_i)), \\ &= \frac{1}{N} \sum_{i=N-n}^{N} (y_i, h(x_i))^2,\end{aligned}$$

   assuming that the data is iid.

6. The students might learn how the algorithm works and use that to their advantage. Let's assume that MLA has a high level of math difficulty. If the student is not good at math, then they could decide to only take courses with low levels of math difficulty, where they get better grades. Since they don't have any courses with high math difficulty, the course with closest the math difficulty to MLA, would be a course with a lower math difficulty. This way they would get a grade corresponding the the lower level math courses that they excel in, even though they are bad at math.

2 Digits Classification with K Nearest Neighbors (40 points)

2.1 Task #1

I have implemented the $K - NN$ function in python using the following code:

```
# Create KNN function
def knn(training_point, training_labels, test_points, test_labels):

    # squared Euclidean distance #(m,n)
    E_dist = (
        np.sum(training_point**2, axis=1)[:, None] - #(m,d)
        2 * training_point @ test_points.T + #(m,d) @ (n,d)^T = (m,n)
        np.sum(test_points**2, axis=1)[None, :] #(n,d)
    )

    # Convert labels: 5 -> -1, 6 -> 1
    training_labels = np.where(training_labels == 5, -1, 1)

    # Sort indexes by Euclidean distance
    nearest_index = np.argsort(E_dist, axis=0)

    # Cumulative sum of the sorted predicted labels
    knn_cumsum = np.cumsum(training_labels[nearest_index], axis=0)

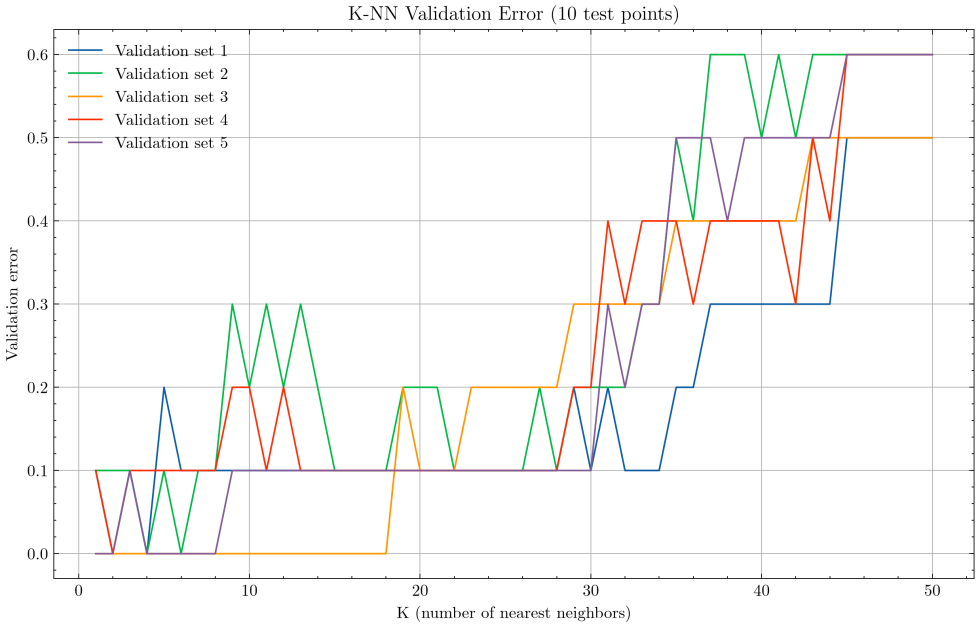
    # Majority vote: if the sum is positive, predict 1 (label 6), else
   predict -1 (label 5)
    knn_pred = np.where(knn_cumsum > 0, 6, 5)

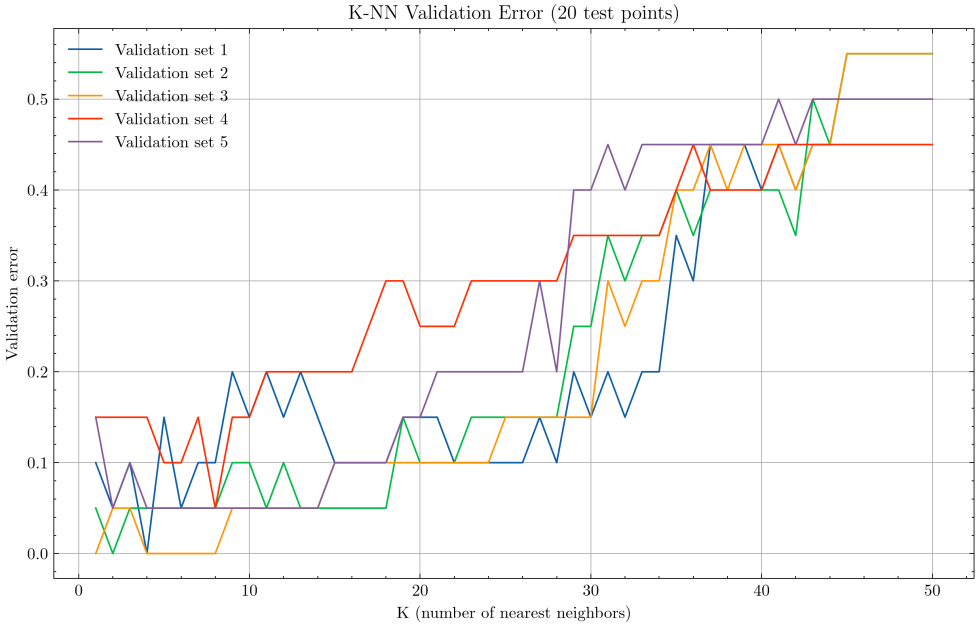
    # Calculate errors
    error_raw = np.mean(knn_pred != test_labels[None,:],axis=1)

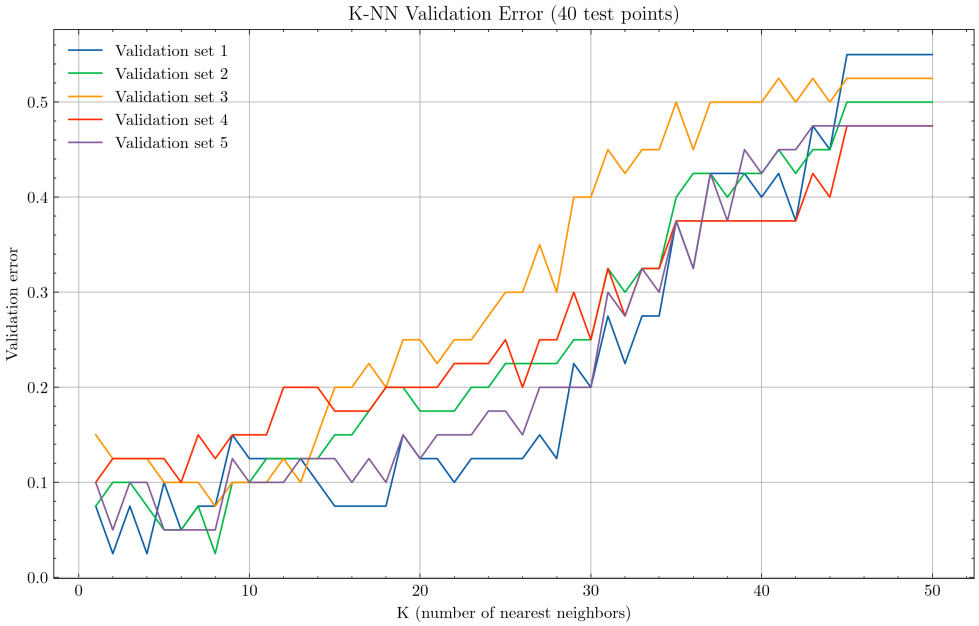
    # Create an array to store the predicted labels for each test point
    k_vals = np.arange(1, len(training_labels)+1)
    error = np.column_stack([k_vals, error_raw])

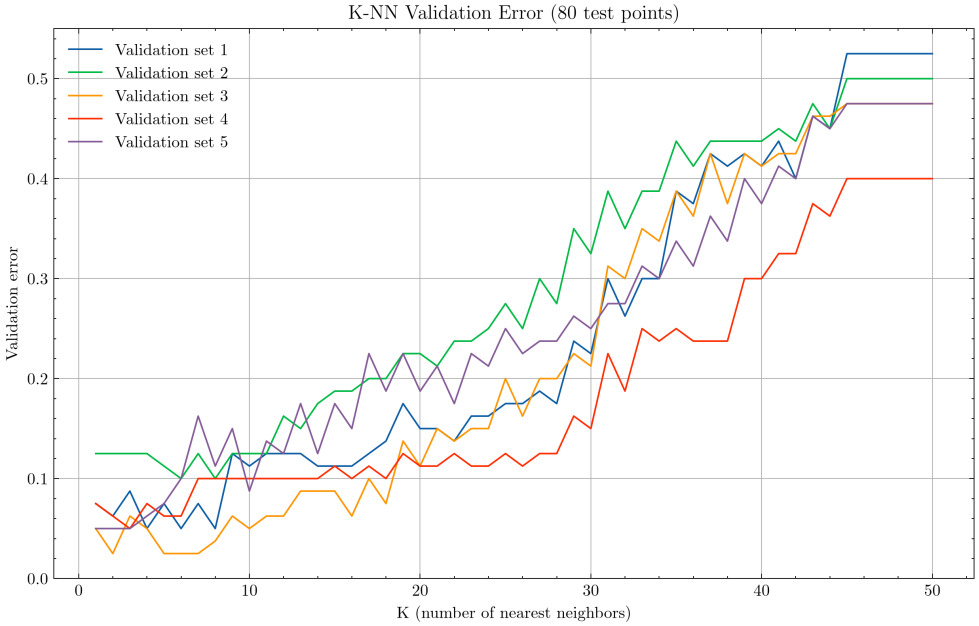

    return error
```

The 4 figures below shows the average validation errors of 5 validation sets $i \in \{1, 2, \ldots, 5\}$ for each of the different sizes of validation sets $n \in \{10, 20, 40, 80\}$.

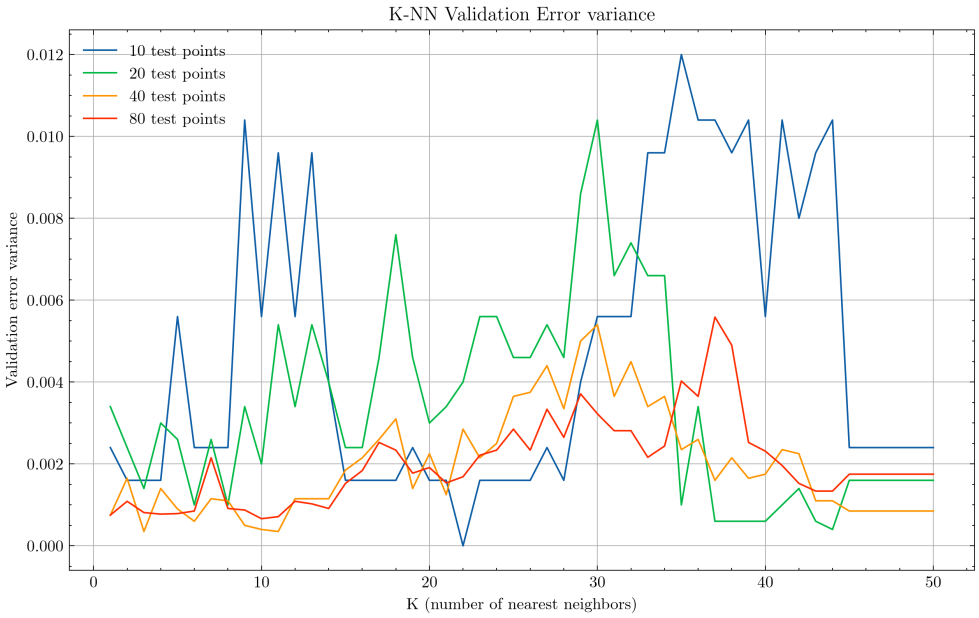

K-NN Validation Error (10 test points)
Validation set 1
Validation set 2
Validation set 3
Validation set 4
Validation set 5
Validation error
K (number of nearest neighbors)
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0
10
20
30
40
50


K-NN Validation Error (20 test points)
Validation set 1
Validation set 2
Validation set 3
Validation set 4
Validation set 5
Validation error
K (number of nearest neighbors)
0.0
0.1
0.2
0.3
0.4
0.5
0
10
20
30
40
50





The figure below shows variance of the validation errors for each of the different sizes of validation sets $n \in \{10, 20, 40, 80\}$ as a function of K.



What can you say about fluctuations of the validation error as a function of n? Answer in the report.

Looking at the figure above, we can see that the fluctuations of the validation errors seem to get smaller as we increase n (number of test points). This makes sense, as the Law of Large Numbers implicates, that the sample mean concentrates more tightly around the true mean.

Looking at the 4 first figures, we see that the validation errors area smallest around $[K = 1, K = 5]$ pointing towards the best model having a relatively low value of K.

3 Regression (50 points)

3.1 Task 1 (5 points)

I have implemented the linear regression model in python using the following code:

```
# Create linear regression function
def LR(y, X):

    # Include a bias term (intercept) in the independent variables
    X = np.column_stack([np.ones(X.shape[0]), X])

    # Calculate the parameter estimates
    w = np.linalg.inv(X.T @ X) @ X.T @ y

    # Calculate predicted values
    y_pred = X @ w

    # Calculate the residuals
```

```
    residuals = y - y_pred

    # Calculate the mean squared error (MSE)
    mse = np.mean(residuals ** 2)

    return w, mse, y_pred
```

3.2 Task 2 (10 points)

Using the linear regression algorithm from above, i get the following parameter estimates: $a \approx 0.2591$, $b \approx 0.0314$, and the mean-squared-error: $MSE \approx 34.8356$. The model i estimated using the linear regression algorithm is:

$$\log(y_i) = ax_i + b.$$

I calculated MSE of $h(x) = \exp(ax + b)$ using:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - h(x_i))^2.$$

3.3 Task 3 (10 points)

It can be shown, that:

$$\arg\min_{a,b} \sum_{i=1}^{N} \left(y_i - \exp\left(ax_i + b\right)\right)^2 \neq \arg\min_{a,b} \sum_{i=1}^{N} \left(\ln\left(y_i\right) - \left(ax_i + b\right)\right)^2,$$

by using an example.

Consider the data $(x_1, x_2) = (0, 0)$ and $(y_1, y_2) = (1, 2)$. This gives me:

$$\arg\min_{a,b} \left(\left(1 - \exp\left(b\right)\right)^2 + \left(2 - \exp\left(b\right)\right)^2\right) \neq \arg\min_{a,b} \left(\left(\ln\left(1\right) - b\right)^2 + \left(\ln\left(2\right) - b\right)^2\right),$$

I know define:

$$\arg\min_{a,b_1} \left(\left(1 - \exp\left(b_1\right)\right)^2 + \left(2 - \exp\left(b_1\right)\right)^2\right),$$

and

$$\arg\min_{a,b_2} \left(\left(\ln\left(1\right) - b_2\right)^2 + \left(\ln\left(2\right) - b_2\right)^2\right),$$

which gives me the FoCs:

$$\begin{aligned}
\frac{\partial\left(\left(1-\exp\left(b_1\right)\right)^2+\left(2-\exp\left(b_1\right)\right)^2\right)}{\partial b_1} &= 0,\\
\Leftrightarrow 2\left(1-\exp\left(b_1\right)\right)\left(-\exp\left(b_1\right)\right)+2\left(2-\exp\left(b_1\right)\right)\left(-\exp\left(b_1\right)\right) &= 0,\\
\Leftrightarrow 2\exp\left(2b_1\right)-3\exp\left(b_1\right) &= 0,\\
\Leftrightarrow 2\exp\left(b_1\right)-3 &= 0,\\
\Leftrightarrow \exp b_1 &= \frac{3}{2},\\
\Leftrightarrow b_1 &= \ln\left(\frac{3}{2}\right),\\
\Leftrightarrow b_1 &\approx 0.41,
\end{aligned}$$

and

$$\begin{aligned}
\frac{\partial\left(\left(\ln\left(1\right)-b_2\right)^2+\left(\ln\left(2\right)-b_2\right)^2\right)}{\partial b_2} &= 0,\\
\Leftrightarrow 2\left(-b_2\right)\left(-1\right)+2\left(\ln\left(2\right)-b_2\right)\left(-1\right) &= 0,\\
\Leftrightarrow 2b_2-2\ln\left(2\right)+2b_2 &= 0,\\
\Leftrightarrow 2b_2=\ln\left(2\right) &= 0,\\
\Leftrightarrow b_2 &= \frac{\ln\left(2\right)}{2},\\
\Leftrightarrow b_2 &\approx 0.35.
\end{aligned}$$

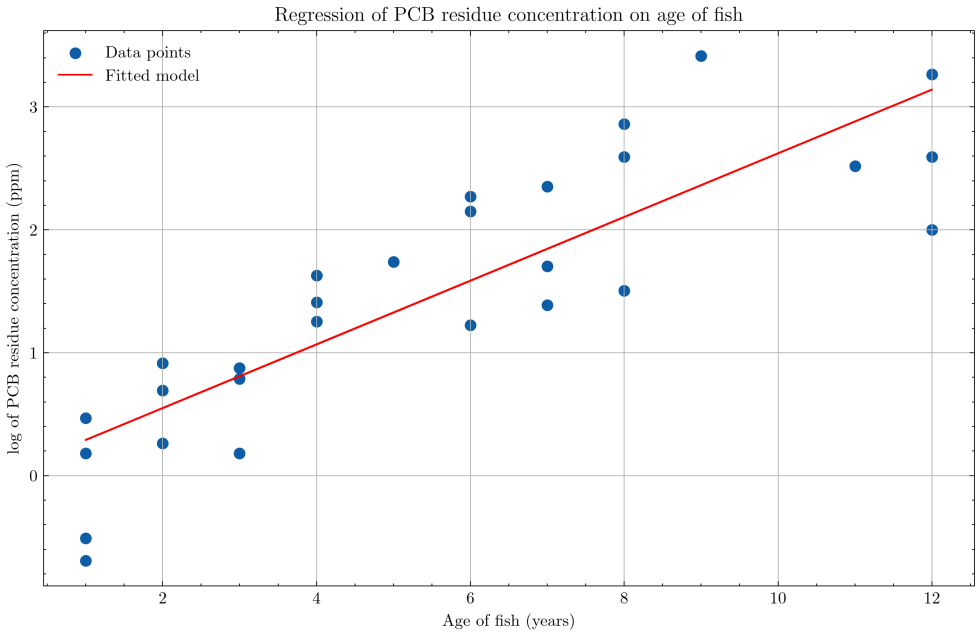
Since $b_1 \approx 0.41 \neq b_2 \approx 0.35$,

$$\arg\min_{a,b}\sum_{i=1}^{N}\left(y_i-\exp\left(ax_i+b\right)\right)^2 \neq \arg\min_{a,b}\sum_{i=1}^{N}\left(\ln\left(y_i\right)-\left(ax_i+b\right)\right)^2,$$

must be true.

3.4 Task 4 (5 points)

The following figure shows a plot of the training data and a line showing the predicted values of PCB (ppm) in fish as a function of the fish's age:



3.5 Task 5 (10 points)

Using the equation:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - h(x_i))^2}{\sum_{i=1}^{n}(y - \bar{y})^2},$$

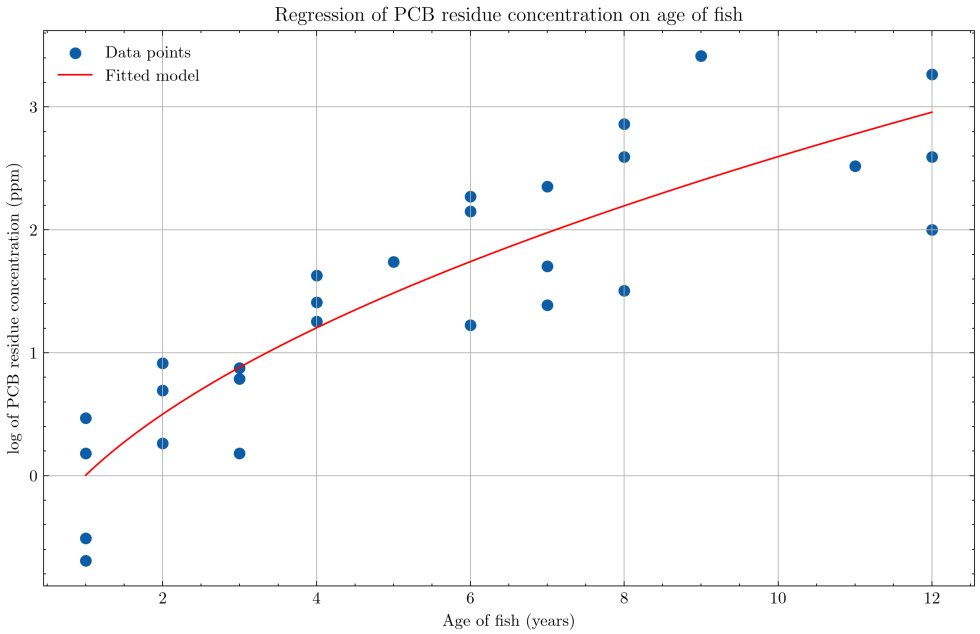
which gave me $R^2 \approx 0.3570$.

R^2 can be interpreted as the amount of the variation in y that the model $h(x) = exp(ax + b)$ can explain. There might be variation in y, that other unused variables can explain (and used to predict y) or just random variation.

if $R^2 = 1$, then the model $h(x) = exp(ax + b)$ explains y perfectly. This means that $y = \bar{(y)} = h(x)$. If $R^2 = 0$, then x has no predictive ability wrt. y. In other words. y is independent of x and $h(x)$ always predicts the mean of y. Note that these things can only be concluded for the interval within the training data i.e. $([\min(y), \max(y)], [\min(x), \max(x)])$.

3.6 Task 6 (10 points)

The figure below shows a plot of the transformed data $(\log(y_i), x_i)$ and a line showing the predicted values of PCB (ppm) in fish as a function of the fish's age, estimated by the linear model: $\log(y_i) = a\sqrt{x_i} + b$:



The mean-squared-error of the non-linear model $y_i = \exp(a\sqrt{x_i} + b)$ is estimated to $MSE \approx 28.0844$. The R^2 value of the model is estimated to $R^2 \approx 0.4816$.

Compared to the former model (Task 2), MSE is now lower ($28.1 < 34.8$), and R^2 is higher ($0.48 > 0.36$). This indicates that the newer model has a better predictive ability of PCB in fish given their age.